Mizan Sales Analyst AI Platform

Architecture and Technical Overview

The Mizan Sales Analyst AI Platform is a multi-agent orchestrated system designed to automatically analyze a company's trial balance (Mizan), extract financial insight, map the trading ecosystem, and generate a strategic, bank-targeted B2B Sales Report.

System Architecture

The core architecture uses a directed agent graph (powered by `langgraph`) that routes unstructured data through a pipeline of logical and analytical steps:

1. **Data Ingestion (Excel Parser)**
    - **Role:** Extracts the raw `.xlsx` Mizan (Trial Balance) document using `pandas`.
    - **Process:** The ingestion agent builds a dynamic mapping mapping of Standard Uniform Chart of Account (TDHP) codes (e.g., *120*, *600*) mapped exactly to what they represent (e.g. *120 Alıcılar*). It calculates the debit, credit, debit balance, and credit balance (Borç, Alacak, Borç Bakiye, Alacak Bakiye) for each account. Finally, it identifies the accounting period of the document.
2. **Quant Analyst (Quantitative Intelligence)**
    - **Role:** Processes raw accounting lines into meaningful financial ratios.
    - **Process:** The agent computes localized financial metrics (Profitability, Turn-over periods, Liquidity ratios, Leverage, Competitor banking share, Check Risk Ratio, Cash Conversion Cycles). It dynamically evaluates both *period movement* and *closing balances*.
    - Once all numbers are evaluated, it constructs a prompt containing the mathematical data and utilizes the LLM (`gemma-3-27b-it`) to interpret what these numbers mean contextually.
3. **Verifier (Validation Checkpoint)**
    - **Role:** Quality assurance gate.
    - **Process:** Acts as a deterministic evaluator for the Quant Analyst. It checks if essential financial nodes (e.g., Revenue 600, COGS 620) successfully triggered expected algorithms. It also evaluates if ratios are within safe boundaries to detect faulty data processing.
    - If validation fails, it triggers an intelligent `retry` back to the Quant Analyst; if it passes, it forwards the state down the pipeline.
4. **Network Mapper (Ecosystem Intelligence)**
    - **Role:** Extracts the supply chain ecosystem of the firm.
    - **Process:** Scans detailed account leaves under 120 (Customers), 320 (Suppliers), 102/300/400 (Competitor Banks). Identifies top players based on transaction volumes.
    - Generates a Graph representation mapping the target firm to its suppliers and customers.
5. **Strategist (Corporate Banking Expert)**
    - **Role:** Generates the actionable, B2B sales pitch document.
    - **Process:** Ingests the quantitative interpretation, the evaluation metrics, and the mapped trading ecosystem into a massive system prompt directed to the LLM (`gemma-3-27b-it`).

- The LLM acts as an experienced corporate banking product manager producing actionable strategies (such as Refinancing Buyout, Cash Management, Supplier Finance solutions). Output format adheres to strict professional branding structures.

6. **Translator & Multi-Lingual Subsystem**
   - **Role:** Ensures the report is available for domestic and international stakeholders.
   - **Process:** Uses the LLM to contextually translate the complexly structured strategy report into banking-fluent Turkish without losing markdown tables, formatting, or financial tone.
7. **Report Generators (HTML & PDF Builder)**
   - **Role:** Post-processing and presentation.
   - **Process:** Takes the generated markdown content from the Strategist and Translator agents. It parses tables, headings, and bold texts, injecting them into `reportlab` canvas elements configured with strict `INGMe` font and brand colors (`#FF6200` Orange, `#000066` Navy). Generates production-ready, beautiful downloadable HTML and PDF files inside the `/output/` directory.

Pipeline Execution

The system is compiled into a standalone Python runtime environment without explicitly requiring complex orchestration middleware. To run the full analysis locally:

```
python3 run_pipeline.py --file YOUR_DATA.xlsx --turkish
```

Optionally, adding `--no-graph` bypasses visually drawing Network Nodes using networkx to quickly execute the backend parsing.